

Universidad de Jaén

Aplicaciones Seguras en la Nube

Tema 4

Actividad 4.1: Diseño y despliegue de una aplicación en la nube con Kubernetes

---

Miguel Bolívar García

Javier Francisco Dibo Gómez

Clúster: Kubernetes local (Docker Desktop, *kind*) · Analizador: KICS v2.2.0

2 de noviembre de 2026

Índice

1. Introducción y entorno de trabajo 1

2. Tarea 1: configuración inicial del despliegue local 1

3. Tarea 2: análisis estático de seguridad 3

4. Tarea 3: ajuste de recursos y configuración dinámica 4

4.1. Gestión de recursos 4

4.2. *ConfigMap* y *Secret* 6

5. Análisis KICS final 7

6. Conclusiones 8

1. Introducción y entorno de trabajo

Esta actividad despliega en un clúster Kubernetes local la aplicación `phpwebappv2` de las actividades del tema 3: dos frontales PHP balanceados y una base de datos MariaDB [1, 2]. Sobre ese despliegue se analizan los manifiestos con el analizador estático KICS [3], se corrigen los hallazgos de severidad crítica y alta, se limitan los recursos de los pods y la configuración se separa de los manifiestos mediante un *ConfigMap* y un *Secret*. Todos los manifiestos están en el directorio `k8s/` del proyecto.

Todas las capturas corresponden al equipo de Miguel Bolívar García (Tabla 1).

Tabla 1: Entorno de trabajo.

Equipo	`PC-MIGUELBG`: Intel Core i9-9900K, 48 GB de RAM
Anfitrión Linux	Ubuntu 26.04 sobre WSL2 (kernel `6.18.54.1-microsoft-standard-WSL2`)
Kubernetes	Docker Desktop con un clúster *kind* de un nodo (`desktop-control-plane`, 16 CPU y unos 23,4 GiB asignables): Kubernetes v1.36.1, *runtime* containerd 2.3.1
Herramientas	`kubectl`; Traefik como controlador Ingress (clase `traefik`); `metrics-server`; KICS v2.2.0; Apache Bench 2.3
Imágenes	`asn/phpwebapp:v1` (Apache 2.4.69 y PHP 8.5.12) y `mariadb:latest`

2. Tarea 1: configuración inicial del despliegue local

El despliegue se compone de tres ficheros (Tabla 2). `mariadb.yaml` es el que proporciona el repositorio de la asignatura, con un único cambio: la etiqueta `app: db` se sustituyó por `run: mariadb`, para dejar libre la clave app, que la actividad 4.2 reserva para `app: asn-app`.

Tabla 2: Manifiestos del despliegue.

Fichero	Recursos	Configuración
`wwwserver.yaml`	*Deployment* y *Service* `wwwserver`	2 réplicas del contenedor `phpwebapp` con la imagen `asn/phpwebapp:v1` en el puerto 80. *Service* de tipo `NodePort`
`mariadb.yaml`	*Deployment* y *Service* `mariadb`	1 réplica de `mariadb:latest`. *Service* `ClusterIP` en el 3306; su nombre es el host de la conexión
`loadbalancer.yaml`	*Ingress* `lb-ingress`	Envía todo lo que llega a `/` al servicio `wwwserver:80`. Lo atiende Traefik

La imagen se indica con su versión (`v1`) y con `imagePullPolicy: IfNotPresent`. Así el clúster usa la imagen construida en local y no intenta descargarla de Docker Hub, donde no existe. Los frontales reciben por variables de entorno el usuario (`MYSQL_USER`), la clave (`MYSQL_PASSWORD`) y, en la versión final, la cadena de conexión (`DB_DSN`). La Sección 4.2 explica cómo pasaron a un *ConfigMap* y un *Secret*.

Tras aplicar los manifiestos, los tres pods están en `Running` sin reinicios, los *Deployments* tienen `2/2` y `1/1` réplicas disponibles, y el *Ingress* `lb-ingress`, de clase `traefik`, tiene como *backends* los dos frontales, `10.244.0.12` y `10.244.0.13` (Figura 1).

```
===== PODS =====
NAME                         READY   STATUS    RESTARTS   AGE     IP            NODE                   NOMINATED NODE   READINESS GATES
mariadb-8dcbb65bb-986ln      1/1     Running   0          16m     10.244.0.9    desktop-control-plane  <none>           <none>
wwwserver-69d77d965-2zmft    1/1     Running   0          3m29s   10.244.0.13   desktop-control-plane  <none>           <none>
wwwserver-69d77d965-jgs29    1/1     Running   0          3m29s   10.244.0.12   desktop-control-plane  <none>           <none>
===== DEPLOYMENTS =====
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
mariadb     1/1     1            1           16m
wwwserver   2/2     2            2           13m
===== SERVICES =====
NAME         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP        7h9m
mariadb      ClusterIP   10.96.207.18    <none>        3306/TCP       16m
wwwserver    NodePort    10.96.249.247   <none>        80:32017/TCP   13m
===== INGRESS =====
NAME         CLASS     HOSTS   ADDRESS      PORTS   AGE
lb-ingress   traefik   *       172.18.0.5   80      12m
===== BACKENDS DEL INGRESS =====
Name:             lb-ingress
Labels:           <none>
Namespace:        default
Address:          172.18.0.5
Ingress Class:    traefik
Default backend:  <default>
Rules:
  Host        Path  Backends
  ----        ----  --------
  *
              /   wwwserver:80 (10.244.0.12:80,10.244.0.13:80)
Annotations:  <none>
Events:       <none>
```

Figura 1: Recursos desplegados en el clúster.

La aplicación responde en `http://localhost/` (Figura 2). La nota *Prueba Kubernetes*, añadida desde el formulario, confirma que los frontales escriben en MariaDB. El pie de la página indica el pod que atendió la petición. En el panel de Traefik (`http://localhost/dashboard/`), el servicio `default-wwwserver-80` aparece en estado *Success* con los dos frontales `UP` y peso 1 (Figura 3).

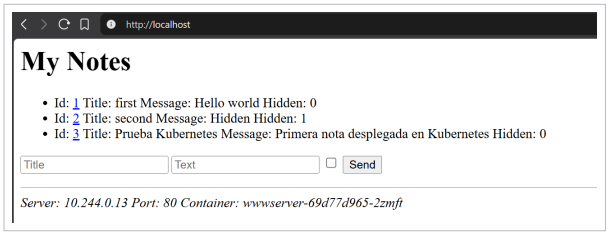


Figura 2: Aplicación en `http://localhost/`.

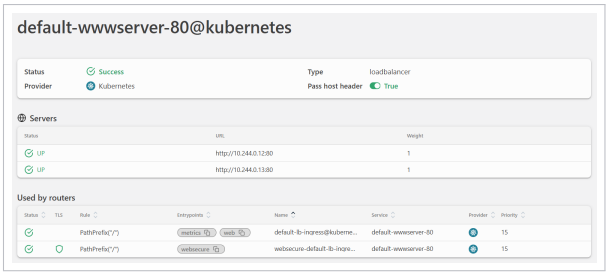


Figura 3: Servicio `wwwserver` en el panel de Traefik.

Para comprobar el reparto se lanzaron diez peticiones con `curl`, cada una con `Connection: close` para abrir una conexión nueva, y se extrajo el contenedor que indica la página. Las respuestas alternan entre los dos pods (Figura 4).

```
miguelbg@PC-MIGUELBG:~/asn/t4/phpwebappv2$ kubectl get pods -l run=wwwserver -o wide
NAME                          READY   STATUS    RESTARTS   AGE   IP            NODE                    NOMINATED NODE   READINESS GATES
wwwserver-69d77d965-2zmft   1/1     Running   0          10m   10.244.0.13   desktop-control-plane   <none>           <none>
wwwserver-69d77d965-jgs29   1/1     Running   0          10m   10.244.0.12   desktop-control-plane   <none>           <none>
miguelbg@PC-MIGUELBG:~/asn/t4/phpwebappv2$ for i in {1..10}; do
  curl -s -H "Connection: close" http://localhost/ | grep -o 'Container: [^<]*'
done
Container: wwwserver-69d77d965-jgs29
Container: wwwserver-69d77d965-2zmft
Container: wwwserver-69d77d965-jgs29
Container: wwwserver-69d77d965-2zmft
Container: wwwserver-69d77d965-jgs29
Container: wwwserver-69d77d965-2zmft
Container: wwwserver-69d77d965-jgs29
Container: wwwserver-69d77d965-2zmft
Container: wwwserver-69d77d965-jgs29
Container: wwwserver-69d77d965-2zmft
```

Figura 4: Diez peticiones repartidas entre los dos frontales.

3. Tarea 2: análisis estático de seguridad

Se ejecutó KICS v2.2.0 [3] sobre el directorio `k8s/`, con informe en HTML. El primer análisis da 47 hallazgos: 0 críticos, 2 altos, 21 medios, 22 bajos y 2 informativos (Figura 5).

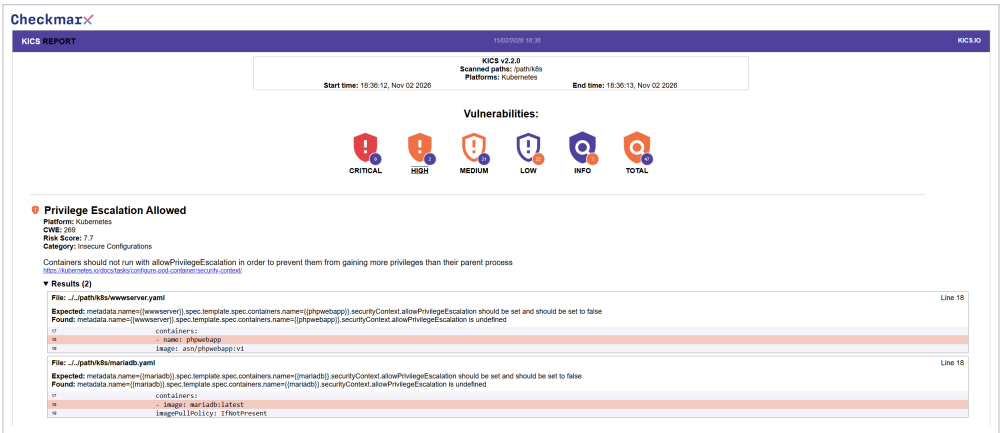


Figura 5: Informe inicial de KICS filtrado por severidad alta.

Los dos hallazgos altos son de la misma regla, *Privilege Escalation Allowed* (CWE-269, *risk score* 7,7, categoría *Insecure Configurations*): ni el contenedor `phpwebapp` de `wwwserver.yaml` ni el contenedor `mariadb` de `mariadb.yaml` definen `securityContext.allowPrivilegeEscalation`. Este campo decide si un proceso del contenedor puede obtener más privilegios que su proceso padre, por ejemplo al ejecutar un binario *setuid*, y controla directamente el indicador `no_new_privs` del kernel [4]. Si no se declara, la escalada está permitida.

El riesgo es real en esta aplicación: `index.php` concatena datos del usuario en consultas SQL y en una llamada a `shell_exec`. Quien explote esas inyecciones ejecutará comandos en el contenedor, y esta opción le cierra una vía de escalada; no corrige la aplicación, pero limita el daño. Se añadió a los dos contenedores:

```
        securityContext:
          allowPrivilegeEscalation: false
```

Al aplicar los cambios, Kubernetes recreó los pods de ambos *Deployments*, que quedaron en `Running` sin reinicios. Dentro de los tres contenedores, `/proc/1/status` muestra `NoNewPrivs: 1`, es decir, la restricción está activa en ejecución, no solo declarada (Figura 6).

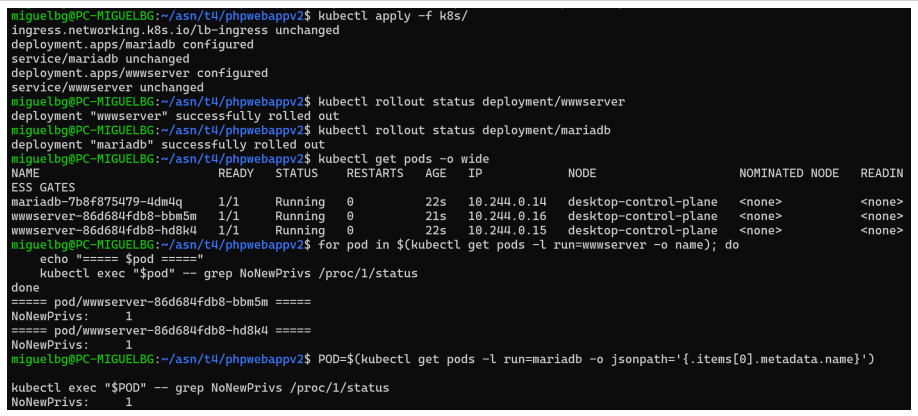


Figura 6: Aplicación de la corrección y comprobación de `NoNewPrivs` en los tres pods.

El nuevo análisis ya no tiene hallazgos altos: 0 críticos, 0 altos, 21 medios, 20 bajos y 2 informativos, 43 en total (Figura 13b). Los bajos pasan de 22 a 20. El informe final ya no incluye la regla *Pod or Container Without Security Context*, lo que es coherente con que ambos contenedores declaren ahora un `securityContext`.

4. Tarea 3: ajuste de recursos y configuración dinámica

4.1. Gestión de recursos

El informe posterior a la tarea 2 mantenía los avisos de la categoría *Resource Management*: cuatro reglas, cada una en los dos contenedores (Tabla 3). Todas tienen el CWE-400 (*Uncontrolled Resource Consumption*): sin límites, un pod puede consumir la CPU o la memoria de todo el nodo y dejar sin servicio al resto.

Tabla 3: Avisos de *Resource Management*.

Regla de KICS	Severidad	Campo que falta
Memory Limits Not Defined	Media	`resources.limits.memory`
Memory Requests Not Defined	Media	`resources.requests.memory`
CPU Limits Not Set	Baja	`resources.limits.cpu`
CPU Requests Not Set	Baja	`resources.requests.cpu`

El *request* es lo que el planificador reserva: el pod solo se coloca en un nodo que lo tenga libre. El *limit* es el máximo: por encima del de CPU se restringe el tiempo de procesador, y por encima del de memoria el kernel termina el contenedor (`OOMKilled`) [5].

Medición inicial. Antes de fijar ningún valor se midió el consumo, como referencia, sobre el despliegue de la tarea 2. En ese momento los pods de la aplicación no tenían *requests* ni *limits* (Figura 7). El nodo del clúster dispone de 16 CPU y 24.565.652 KiB (unos 23,4 GiB) de memoria asignables. Fuera de la prueba de carga consumía 128m de CPU y 1186 MiB (4 %), y el resto de pods del clúster (componentes de Kubernetes y `metrics-server`) reservaban 1050m de CPU y 490 MiB.

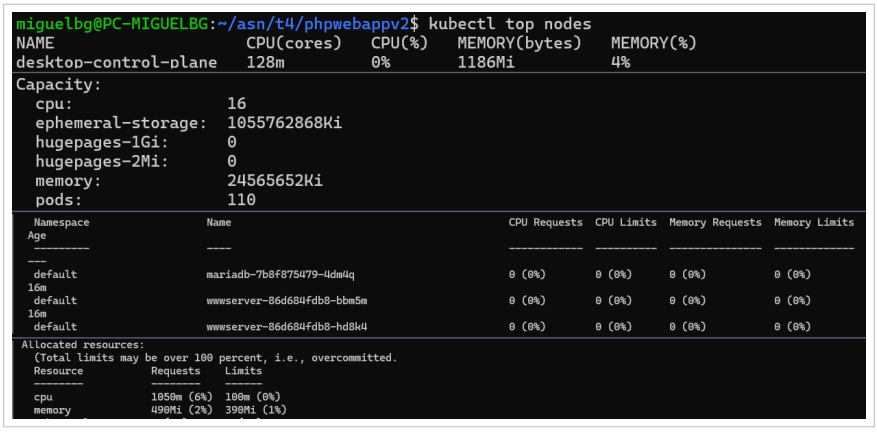


Figura 7: Extracto de `kubectl top nodes` y `kubectl describe nodes` antes de fijar los recursos.

Para medir el consumo con carga se lanzó Apache Bench contra el *Ingress* con 20 conexiones simultáneas durante 30 segundos, mientras `kubectl top pod` se refrescaba cada 5 segundos en otra terminal (Figura 8):

```
ab -c 20 -t 30 http://localhost/
```

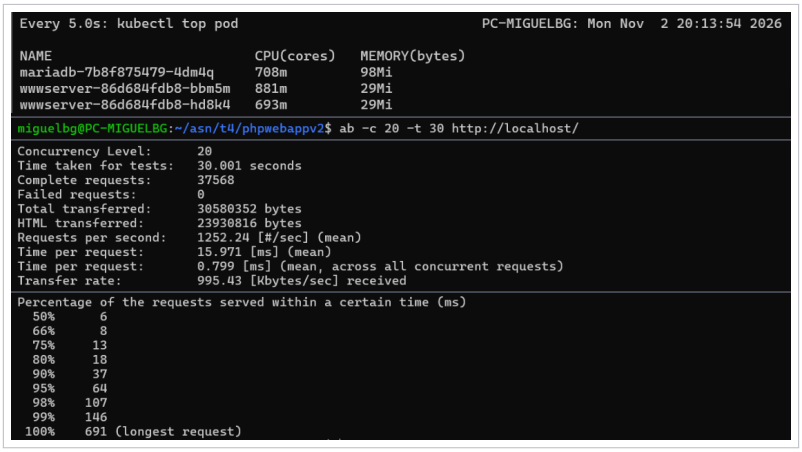


Figura 8: Extracto de las dos terminales durante la prueba inicial: `kubectl top pod` y resultado de `ab`.

La prueba completó 37.568 peticiones sin ningún fallo, a unas 1.252 peticiones por segundo, con un tiempo medio de 15,97 ms por petición. El 95 % se atendió en 64 ms o menos, el 99 % en 146 ms o menos y la más lenta tardó 691 ms. La Tabla 4 recoge el consumo de cada pod durante esa prueba junto a los valores que se configuraron después.

Tabla 4: Consumo medido en la prueba inicial, sin límites, y recursos configurados después.

	Medido (sin límites)		*Request*		*Limit*	
Pod	**CPU**	**Memoria**	**CPU**	**Memoria**	**CPU**	**Memoria**
`phpwebapp`, réplica 1	881m	29 Mi	100m	64 Mi	1000m	256 Mi
`phpwebapp`, réplica 2	693m	29 Mi	100m	64 Mi	1000m	256 Mi
`mariadb`	708m	98 Mi	250m	128 Mi	1000m	256 Mi
Total configurado			450m	256 Mi	3000m	768 Mi

Valores configurados. Cada contenedor recibió en su bloque `resources` los valores de la Tabla 4. El nodo local tiene recursos de sobra, así que los valores se eligieron pensando en el despliegue de producción, que según el enunciado no debe superar 1 GB de memoria. El límite de CPU de 1000m queda por encima del consumo medido en los tres pods (entre 693m y 881m), y el de memoria de 256 Mi deja un margen amplio sobre los 29 Mi de cada frontal y los 98 Mi de MariaDB. El `Dockerfile` limita Apache a 30 procesos (`MaxRequestWorkers`), lo que acota la memoria de cada frontal. Los *requests* reservan un consumo base, no el pico: con carga, cada pod puede usar CPU libre hasta su límite. La suma de los límites de memoria, 768 MiB, cumple ese máximo. La prueba de carga no se repitió con los límites aplicados, así que su comportamiento con carga y límites no está medido.

Tras aplicar los límites, el análisis queda en 0 críticos, 0 altos, 17 medios, 16 bajos y 2 informativos (Figura 13c). Desaparecen 4 avisos medios y 4 bajos, los de la Tabla 3, y no queda ninguno de *Resource Management*.

4.2. *ConfigMap* y *Secret*

La configuración se separó de los manifiestos según su sensibilidad. El *ConfigMap* `phpwebapp-config` (`configmap.yaml`) guarda los datos no confidenciales: `DB_DSN` (`mysql:host=mariadb;dbname=notesdb`), `DB_NAME` y `MYSQL_USER`. Las contraseñas, `MYSQL_PASSWORD` y `MYSQL_ROOT_PASSWORD`, van en el *Secret* `db-credentials`, que se creó a partir de `k8s/db.env` antes de aplicar los manifiestos:

```
kubectl create secret generic db-credentials --from-env-file=k8s/db.env
kubectl apply -f k8s/
```

Así ningún manifiesto contiene contraseñas. Los dos *Deployments* obtienen cada variable con `valueFrom`: `configMapKeyRef` para la configuración y `secretKeyRef` para las claves. Por ejemplo, en `wwwserver.yaml`:

```
        - name: MYSQL_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: MYSQL_PASSWORD
```

MariaDB toma `MYSQL_DATABASE` de `DB_NAME` y comparte con los frontales `MYSQL_USER` y `MYSQL_PASSWORD`, de modo que la base de datos y la aplicación no pueden quedar con credenciales distintas. `index.php` pasó a leer también la cadena de conexión del entorno, y se reconstruyó la imagen `asn/phpwebapp:v1`, porque en Kubernetes el pod ejecuta el código copiado en la imagen:

```
$dsn    = $_ENV["DB_DSN"];
$dbuser = $_ENV["MYSQL_USER"];
$dbpass = $_ENV["MYSQL_PASSWORD"];
```

Las contraseñas no se guardan en el *ConfigMap* porque este recurso no ofrece confidencialidad ni cifrado; la documentación de Kubernetes indica usar un *Secret* para los datos confidenciales [6]. Un *Secret* permite restringir su lectura por separado, y `kubectl describe` solo muestra su tamaño (Figura 9). Aun así, codificar en Base64 no es cifrar: quien pueda leer el *Secret* lo decodifica, y por defecto se almacena sin cifrar en `etcd` [7].

```
========== 1. CONFIGMAP ==========
NAME              DATA   AGE
phpwebapp-config  3      10m

========== 3. DETALLES DEL SECRET ==========
Name:         db-credentials
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
MYSQL_PASSWORD:       7 bytes
MYSQL_ROOT_PASSWORD:  6 bytes

========== 4. ESTADO DE LOS PODS ==========
NAME                         READY   STATUS    RESTARTS   AGE
mariadb-dc5dccdfc-kdnf9      1/1     Running   0          8m15s
wwwserver-7589fc4895-2zmjq   1/1     Running   0          8m15s
wwwserver-7589fc4895-q2mcz   1/1     Running   0          8m14s
```

Figura 9: Extracto: *ConfigMap*, *Secret* y pods tras aplicar los cambios.

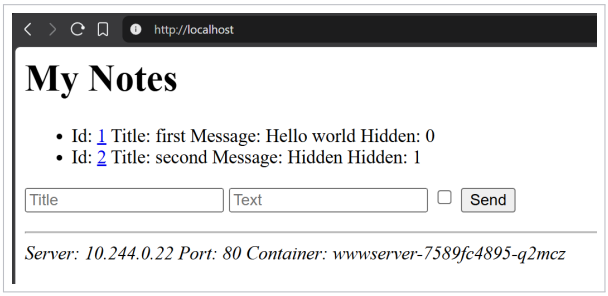


Figura 10: Aplicación funcionando con la configuración externa.

Tras aplicar los cambios, los tres pods están en `Running` sin reinicios (Figura 9) y la aplicación sigue respondiendo (Figura 10). La nota añadida en la tarea 1 ya no aparece porque el pod de MariaDB se recreó y la base de datos no tiene volumen persistente. Las variables llegan a los contenedores con los valores del *ConfigMap* (Figura 11).

```
miguelbg@PC-MIGUELBG:~/asn/t4/phpwebappv2$ POD=$(kubectl get pods -l run=wwwserver -o jsonpath='{.items[0].metadata.name}')

kubectl exec "$POD" -- printenv DB_DSN MYSQL_USER
mysql:host=mariadb;dbname=notesdb
usuario
miguelbg@PC-MIGUELBG:~/asn/t4/phpwebappv2$ POD_DB=$(kubectl get pods -l run=mariadb -o jsonpath='{.items[0].metadata.name}')

kubectl exec "$POD_DB" -- printenv MYSQL_DATABASE MYSQL_USER
notesdb
usuario
```

Figura 11: Variables de entorno dentro de un frontal y del pod de MariaDB.

5. Análisis KICS final

El último análisis, incluido en la entrega (`kics-report/final.html`), da 0 críticos, 0 altos, 18 medios, 19 bajos y 2 informativos, 39 hallazgos en total (Figuras 12 y 13d). Se cumplen los dos objetivos de corrección del enunciado: no queda ningún hallazgo crítico ni alto, y el informe no contiene ninguno de la categoría *Resource Management*. La Tabla 5 resume la evolución.

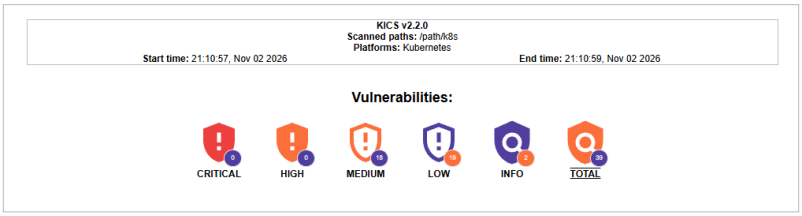
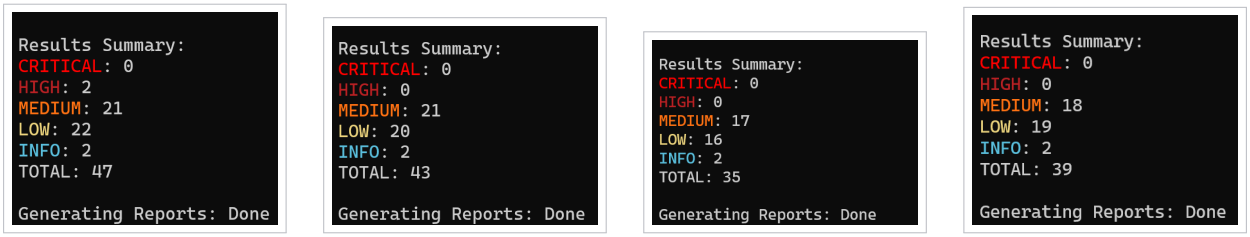


Figura 12: Informe HTML final de KICS.



(a) Inicial (b) Tras corregir los altos (c) Tras fijar los recursos (d) Final

Figura 13: Resúmenes de KICS en la terminal a lo largo de la actividad.

Tabla 5: Evolución de los hallazgos de KICS.

Análisis	Crítico	Alto	Medio	Bajo	Info	Total
(a) Inicial (tarea 1)	0	2	21	22	2	47
(b) Tras corregir los altos (tarea 2)	0	0	21	20	2	43
(c) Tras fijar *requests* y *limits*	0	0	17	16	2	35
(d) Final, con *ConfigMap* y *Secret*	0	0	18	19	2	39

El último paso sube el total de 35 a 39 aunque mejora la configuración. El aviso medio nuevo es *Using Unrecommended Namespace* sobre `configmap.yaml`, creado en el *namespace* `default` como el resto. Los tres bajos nuevos son *Secrets As Environment Variables* (CWE-526), uno por cada variable leída del *Secret*: la regla recomienda montar los secretos como ficheros, porque las variables de entorno pueden acabar en registros o volcados.

Los avisos medios y bajos restantes no son un requisito de corrección del enunciado. Son sobre todo medidas de endurecimiento adicional: contenedores ejecutados como root, *capabilities* como `NET_RAW` sin eliminar, sin perfiles *seccomp* ni AppArmor, sin sondas de disponibilidad, imágenes sin *digest* y `mariadb:latest` sin versión fija. Algunos exigen adaptar las imágenes: Apache arranca como root para abrir el puerto 80 y después cambia a `www-data`. Por último, KICS solo procesó los cuatro manifiestos YAML: `db.env`, con las contraseñas, queda fuera del análisis.

6. Conclusiones

La aplicación del tema 3 funciona en Kubernetes descrita solo con manifiestos declarativos: dos frontales, la base de datos y un *Ingress* que Traefik reparte petición a petición entre los dos pods. El análisis estático detectó una configuración de severidad alta, presente también en el fichero de MariaDB proporcionado, y su corrección se comprobó en ejecución con `NoNewPrivs`, no solo con un nuevo análisis. Los *requests* y *limits* acotan el consumo dentro del presupuesto de 1 GB, y el *ConfigMap* y el *Secret* dejan los manifiestos sin contraseñas.

El análisis estático tiene límites. KICS evalúa reglas sobre los ficheros que procesa: no ve `db.env` ni el código de la aplicación, cuyas inyecciones SQL y de comandos siguen presentes. Y el número de hallazgos no mide la seguridad: el total subió de 35 a 39 con un cambio que mejora la gestión de credenciales. Los resultados deben revisarse uno a uno, no como un contador.

Referencias

[1] José Ramón Balsas Almagro. Actividades Tema 4: Configuración y Despliegue de Aplicaciones en la Nube. Aplicaciones Seguras en la Nube, Universidad de Jaén, 2026. Versión 1.0, octubre de 2026.

[2] José Ramón Balsas Almagro. asnwebapps: contenedores con aplicaciones web de ejemplo, 2026. URL https://github.com/jrbalsas/asnwebapps.

[3] Checkmarx. KICS: Keeping Infrastructure as Code Secure, 2026. URL https://kics.io.

[4] The Kubernetes Authors. Configure a Security Context for a Pod or Container, 2026. URL https://kubernetes.io/docs/tasks/configure-pod-container/security-context/.

[5] The Kubernetes Authors. Resource Management for Pods and Containers, 2026. URL https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/.

[6] The Kubernetes Authors. ConfigMaps, 2026. URL https://kubernetes.io/docs/concepts/configuration/configmap/.

[7] The Kubernetes Authors. Secrets, 2026. URL https://kubernetes.io/docs/concepts/configuration/secret/.